Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей
Кафедра: Электронных вычислительных машин
Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО. КАЛЬКУЛЯТОР ЧИСЛЕННОСТИ НАСЕЛЕНИЯ»
Вариант 2

Студент: М.Н. Дешко

Преподаватель: С.С. Силич

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить основные возможности интегрированной среды разработки IntelliJ IDEA при создании объектно-ориентированного приложения на языке Java, освоить запуск и отладку программы с графическим интерфейсом, а также применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 7 необходимо разработать утилиту для расчета численности населения. Для выполнения работы поставлены следующие задачи:

1. Создать и настроить проект Java в IntelliJ IDEA, изучить средства редактирования, сборки, запуска и отладки;
2. Подобрать средства IntelliJ IDEA для работы с UML и SysML v2 и отразить результат настройки в отчёте;
3. Разработать графическое приложение с главной формой и отдельным окном ввода сигналов;
4. Обеспечить выбор русского или латинского алфавита, сохранение последних корректных данных и их восстановление при повторном вводе;
5. Реализовать проверку входных данных и вывод понятного сообщения при ошибке;
6. Разделить приложение на модель, представление и контроллер и реализовать активное уведомление представления моделью;
7. Проверить основные сценарии работы программы и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Интегрированная среда разработки объединяет редактор исходного кода, средства навигации и рефакторинга, систему сборки, запуск приложения, отладчик, терминал и интеграцию с системой контроля версий. IntelliJ IDEA анализирует Java-код во время редактирования, показывает ошибки и предупреждения, предлагает быстрые исправления и позволяет запускать класс с методом main непосредственно из редактора.

2.2 Редактирование и навигация в IntelliJ IDEA

Редактор IntelliJ IDEA поддерживает автодополнение, импорт классов, форматирование, безопасное переименование и поиск использований. Для перехода к объявлению используется Ctrl+B, для поиска использований – Alt+F7, для форматирования – Ctrl+Alt+L, а для вызова намерений и быстрых исправлений – Alt+Enter. Структура проекта отображается в окне Project, сообщения компилятора и программы – в окнах Build и Run.

2.3 Сборка, запуск и отладка Java-приложения

Исходные файлы компилируются компилятором javac в байт-код JVM. В IntelliJ IDEA проект собирается командой Build Project, а конфигурация запуска класса App выполняется командами Run и Debug. Точка останова устанавливается щелчком в поле слева от строки или сочетанием Ctrl+F8. Во время остановки окно Debug показывает стек вызовов и значения переменных. Команды Step Over, Step Into и Step Out позволяют выполнять программу пошагово.

2.4 Паттерн MVC

MVC разделяет приложение на три части. Model хранит состояние и выполняет предметную логику. View формирует интерфейс и отображает данные. Controller обрабатывает действия пользователя и передаёт команды модели. Такое разделение уменьшает связанность классов и позволяет проверять декодирование независимо от Swing-интерфейса.

2.5 Активная и пассивная модель

В варианте с пассивной моделью контроллер после изменения данных сам инициирует обновление представления. Активная модель сообщает подписчикам об изменении состояния. В разработанном приложении PopulationModel использует паттерн «Наблюдатель» (Observer) через внутренний интерфейс ModelListener.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработана утилита, которая принимает дату (день, месяц, год) и вычисляет прогнозируемую численность населения мира. На главной форме размещена кнопка «Ввести данные», открывающая модальное окно InputDialog. Приложение использует экспоненциальную модель роста населения от базового значения 2020 года. Реализована поддержка восстановления данных: при открытии окна ввода подставляются последние успешно обработанные значения.

3.2 Настройка среды и расширений для UML и SysML v2

В Visual Studio открыт проект C++ с использованием набора инструментов v143 и стандарта C++17. Точка входа – функция main. Для UML применяется плагин PlantUML Integration.

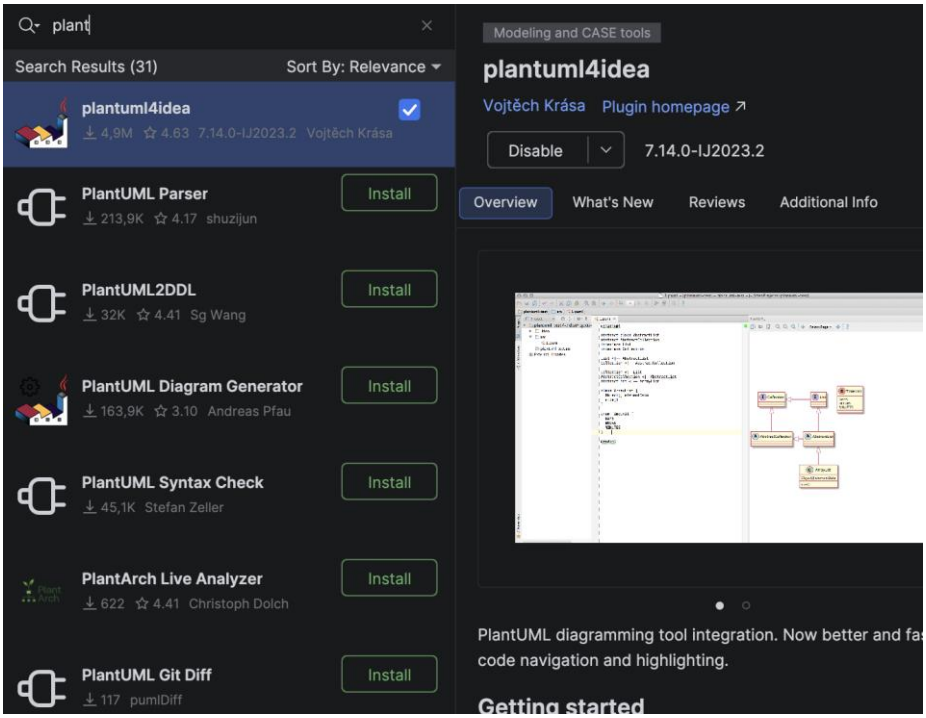


Рисунок 3.1 – Средства IntelliJ IDEA для работы с UML и SysML v2

Структурное представление системы в нотации, близкой к SysML v2:

```
package AstrologerApp {
    part def AstroModel;
    part def MainView;
    part def InputDialog;
    part def AstroController;
    part application {
    part model : AstroModel;
```

```
            part view : MainView;
            part dialog : InputDialog;
            part controller : AstroController;
            connect view to controller;
            connect controller to model;
            connect model to view;
        }
    }
```

3.3 Демонстрация отладчика

Работа отладчика демонстрируется в методе setBirthDate класса AstroModel. Точка останова устанавливается на строке проверки високосного года или вычисления дня недели. В окне *Локальные* можно проследить, как переменные day_, month_, year_ влияют на итоговый результат, и проверить точность вычислений с помощью окна *Контрольные значения (Watch)*.

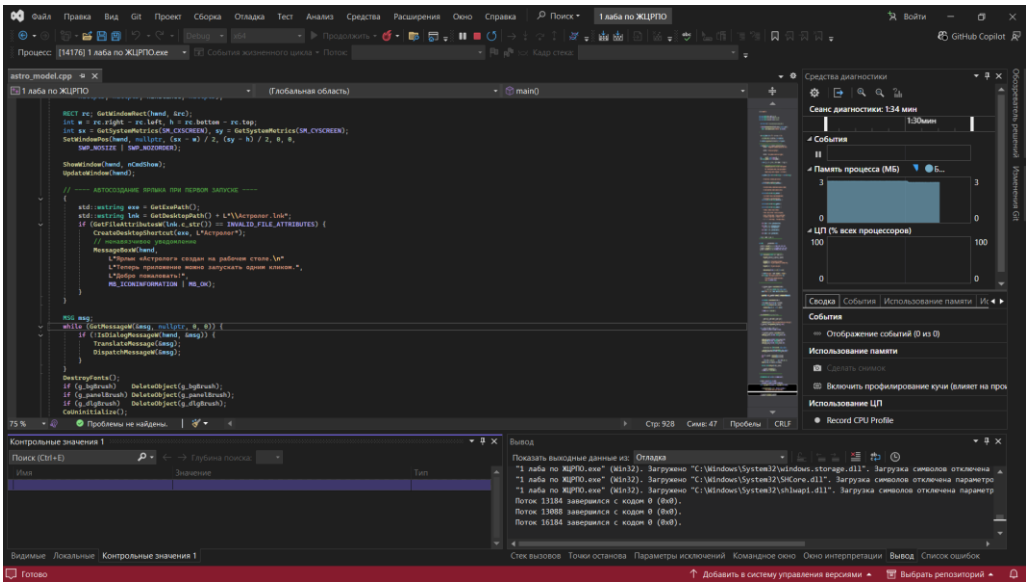


Рисунок 3.2 – Контрольные значения

3.4 Структура проекта и каркас MVC с активной моделью

- Проект разделён на следующие компоненты: AstroModel – хранит дату, флаг валидности, день недели и текст судьбы. Содержит бизнес-логику вычислений и механизм уведомления слушателей (std::vector<Observer>).
- MainWndProc (MainView) – главный интерфейс, подписывается на модель через g_model->subscribe(&UpdateMainWindow) для автоматического обновления текстовых меток.
- InputDialogProc – модальное окно для сбора даты с кастомной отрисовкой элементов управления.
- AstroController – обрабатывает ввод пользователя, выполняет парсинг строк в числа, сохраняет данные в модель и управляет функциями сохранения файла и создания ярлыка.

- main – точка входа, инициализирующая COM-библиотеки, классы и оконные процедуры.
-

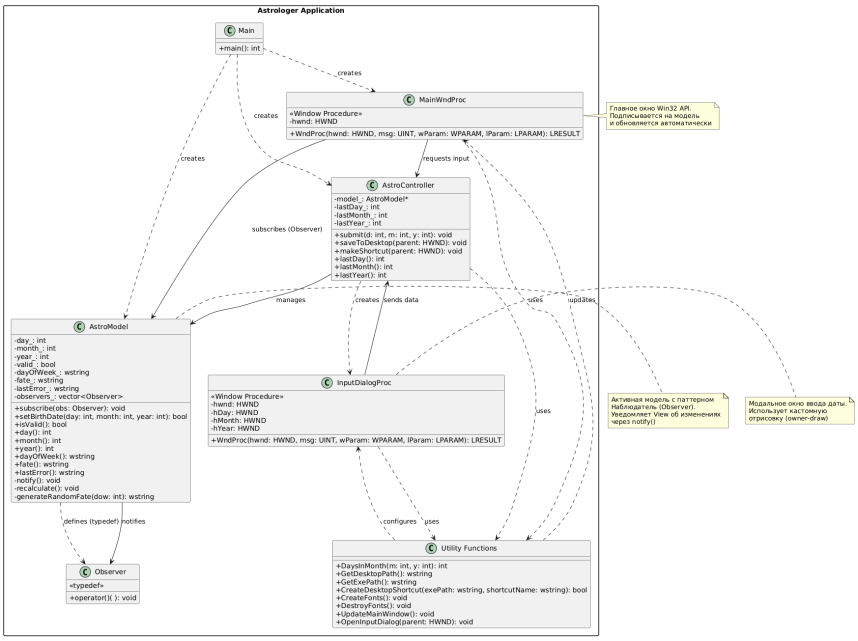


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

AstroModel реализует активную модель. Метод setBirthDate выполняет многоуровневую проверку: диапазоны годов (1900–2100), корректность месяца и дня (с учётом високосных лет через DaysInMonth), а также запрет на даты из будущего. После валидации вызывается recalculate(), определяющая день недели и генерирующая текст судьбы, после чего вызывается notify() для обновления View

3.6 Представление

Главное окно (MainWndProc) отображает текущую дату, день недели и рассчитанное послание судьбы. Благодаря подписке на модель, функция UpdateMainWindow обновляет текстовые метки (SetWindowTextW) мгновенно после того, как модель зафиксировала изменения. Окно ввода (InputDialogProc) обеспечивает ввод даты с кастомной тёмной темой, реализуемой через обработку сообщений WM_CTLCOLORSTATIC, WM_CTLCOLOREDIT и WM_DRAWITEM.

3.7 Контроллер

AstroController управляет жизненным циклом ввода и отвечает за безопасный перевод пользовательского ввода из строкового формата в числовой (_wtoi). В случае некорректных данных (нули в полях или ошибки

валидации модели) контроллер перехватывает ситуацию и выводит сообщение пользователю через MessageBoxW. Также контроллер управляет записью отчёта в UTF-8 файл на рабочий стол и созданием COM-ярлыка (IShellLinkW)

3.8 Запоминание данных и обработка ошибок

Приложение предотвращает ввод некорректных дат. Сообщения об ошибках (например, «Такой даты не существует» или «Дата рождения не может быть в будущем») формируются в модели и передаются через контроллер во View. Восстановление данных реализовано через чтение lastDay(), lastMonth(), lastYear() из контроллера при создании окна InputDialog.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В Visual Studio создано и проверено C++ Win32-приложение. Программа вычисляет день недели и генерирует предсказание, сохраняет последнее успешное состояние для повторного ввода, формирует текстовый отчёт на рабочем столе и создаёт ярлык. Архитектура соответствует MVC с активной моделью: контроллер обрабатывает пользовательское действие, модель выполняет валидацию и уведомляет представление через механизм Observer (std::function).

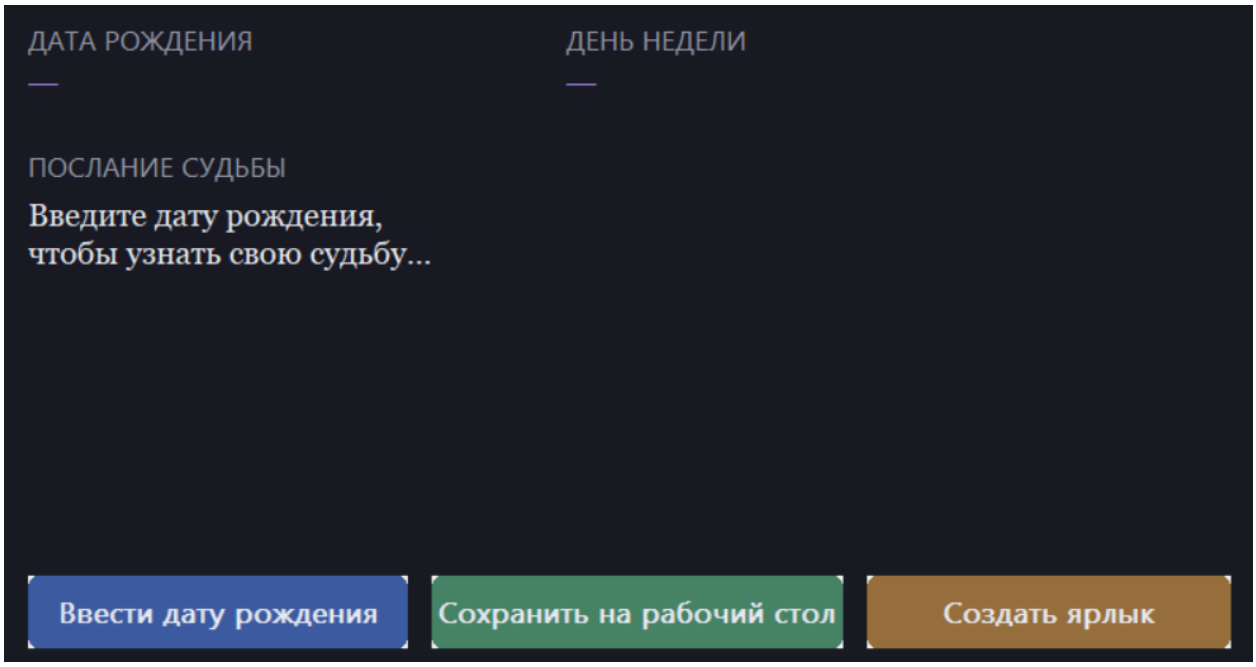


Рисунок 4.1 – Главная форма после запуска

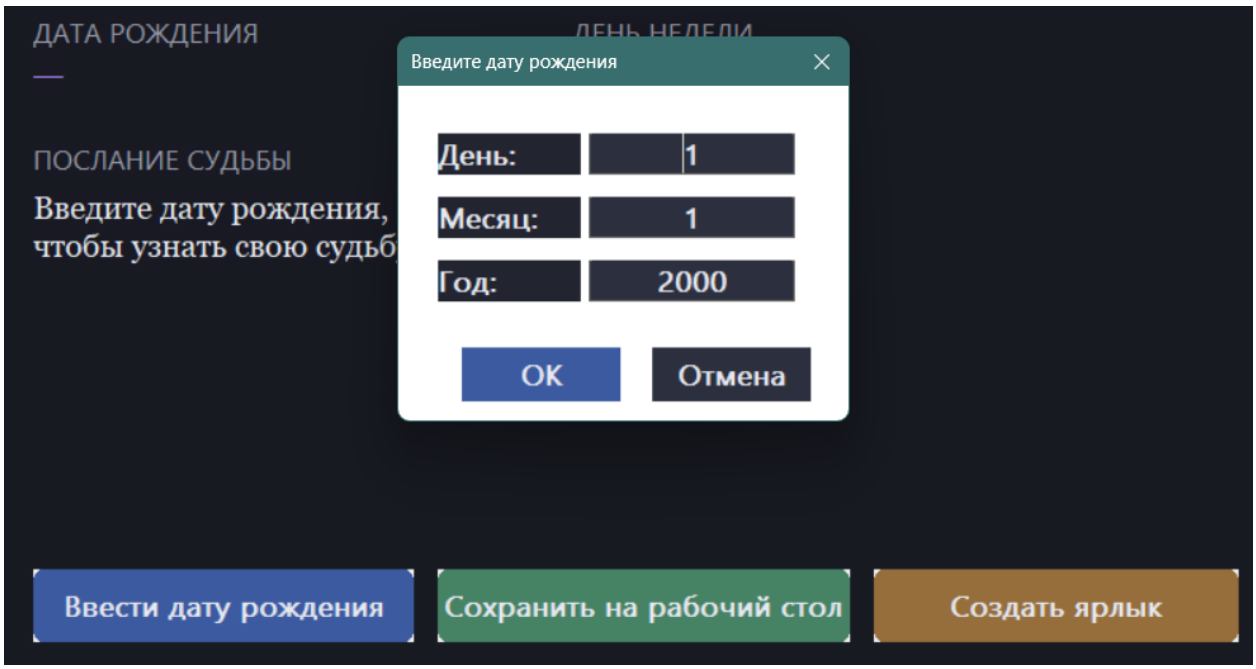


Рисунок 4.2 – Окно ввода данных

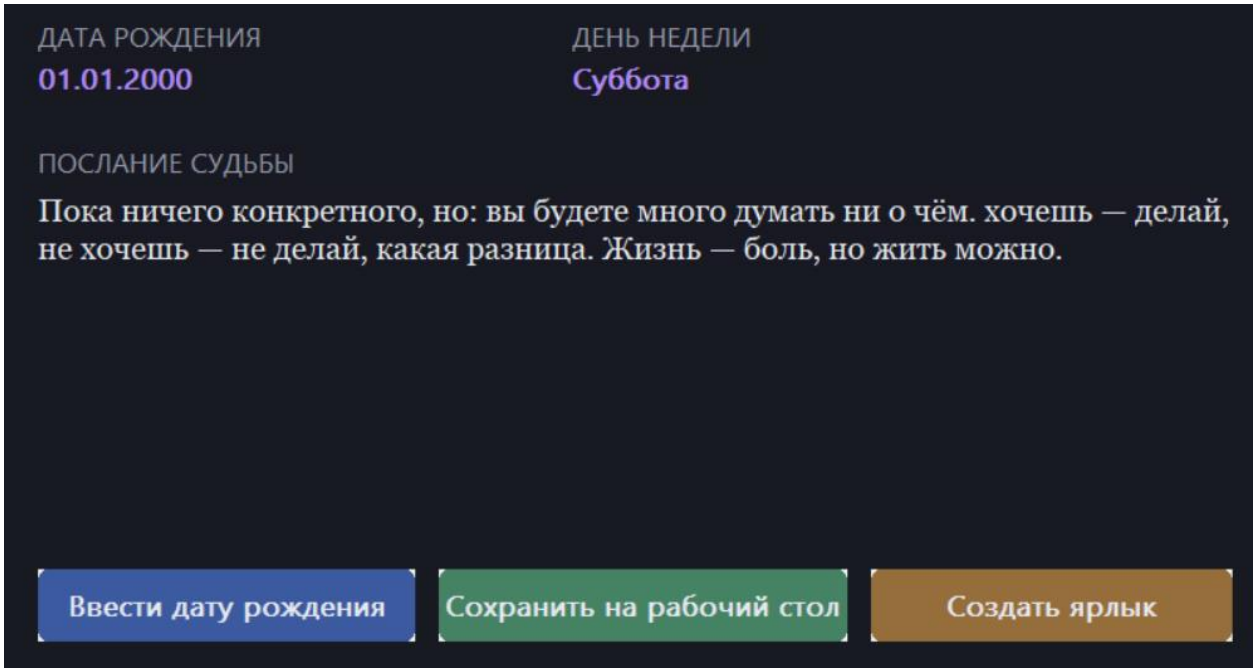


Рисунок 4.3 – Результат вычисления

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, компилятор или интерпретатор, система сборки и запуска, отладчик, средства навигации и рефакторинга, терминал, интеграция с системой контроля версий и механизм расширений. В IntelliJ IDEA основные рабочие окна называются Project, Build, Run, Debug, Problems и Terminal.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение, быстрые исправления, автоматический импорт, форматирование, переход к объявлению, поиск использований, переименование, просмотр иерархии классов и анализ предупреждений. Эти операции позволяют изменять код быстрее и уменьшают вероятность механических ошибок.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

IntelliJ IDEA компилирует Java-файлы командой Build Project. Для запуска выбирается класс App с методом public static void main(String[] args) и выполняется команда Run. При запуске в режиме Debug программа работает под управлением отладчика и останавливается на заданных точках. Проект также можно собрать командой javac --release 17 и запустить командой java с указанием класса by.bsuir.morse.App.

Вопрос 4 Что такое MVC?

MVC – архитектурный паттерн, который разделяет данные и предметную логику, графический интерфейс и обработку действий пользователя. В данной работе роль модели выполняет MorseDecoderModel, роль представления – MainView и InputDialog, а роль контроллера – MorseController.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление, поэтому после её изменения контроллер должен отдельно обновить интерфейс. Активная модель публикует событие при изменении своего состояния. В приложении MorseDecoderModel вызывает firePropertyChange, а MainView получает новое состояние через метод propertyChange и самостоятельно обновляет элементы формы.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности Visual Studio для редактирования, сборки, запуска и отладки C++-приложения. Рассмотрены средства работы с UML и SysML v2 и подготовлена диаграмма компонентов приложения.

Разработана утилита «Астролог» с графическим интерфейсом на базе Win32 API. Программа поддерживает строгую валидацию дат, генерацию случайных предсказаний и особое статическое послание для заданной даты. Компоненты разделены по паттерну MVC, а активная модель уведомляет представление о новом состоянии с помощью паттерна Наблюдатель (Observer), что обеспечивает слабую связанность кода. Реализованы дополнительные функции: сохранение отчёта в UTF-8 и автоматическое создание ярлыка на рабочем столе через COM-интерфейсы.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода приложения

```
#include <windows.h>
#include <commctrl.h>
#include <shlobj.h>
#include <shobjidl.h>
#include <string>
#include <vector>
#include <functional>
#include <ctime>
#include <fstream>
#include <random>

#pragma comment(lib, "comctl32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "ole32.lib")

// ---------- Палитра ----------
static const COLORREF CLR_BG = RGB(24, 26, 34);
static const COLORREF CLR_PANEL = RGB(34, 37, 48);
static const COLORREF CLR_PANEL_2 = RGB(44, 48, 62);
static const COLORREF CLR_TEXT = RGB(230, 232, 240);
static const COLORREF CLR_TEXT_DIM = RGB(150, 155, 170);
static const COLORREF CLR_ACCENT = RGB(120, 170, 255);
static const COLORREF CLR_ACCENT_2 = RGB(180, 140, 255);
static const COLORREF CLR_BTN = RGB(60, 90, 160);
static const COLORREF CLR_BTN_HOVER = RGB(80, 115, 195);
static const COLORREF CLR_BTN_SAVE = RGB(70, 130, 100);
static const COLORREF CLR_BTN_LINK = RGB(150, 110, 60);
```

```
// ---------- ID контролов ----------
enum {
    ID_BTN_INPUT = 1001,
    ID_BTN_SAVE,
    ID_BTN_SHORTCUT,
    ID_EDIT_DAY, ID_EDIT_MONTH, ID_EDIT_YEAR,
    ID_BTN_OK, ID_BTN_CANCEL
};

// ---------- Утилиты ----------
static int DaysInMonth(int m, int y) {
    static const int d[] = { 31,28,31,30,31,30,31,31,30,31,30,31 };
    if (m < 1 || m > 12) return 0;
    if (m == 2) {
        bool leap = (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
        return leap ? 29 : 28;
    }
    return d[m - 1];
}

static std::wstring GetDesktopPath() {
    PWSTR path = nullptr;
    std::wstring result;
    if (SUCCEEDED(SHGetKnownFolderPath(FOLDERID_Desktop, 0,
nullptr, &path))) {
        result = path;
        CoTaskMemFree(path);
    }
    else {
        wchar_t buf[MAX_PATH];
```

```
            DWORD n = GetEnvironmentVariableW(L"USERPROFILE", buf,
MAX_PATH);
            if (n > 0) result = std::wstring(buf) + L"\\Desktop";
        }
        return result;
    }

    static std::wstring GetExePath() {
        wchar_t buf[MAX_PATH];
        DWORD n = GetModuleFileNameW(nullptr, buf, MAX_PATH);
        return std::wstring(buf, n);
    }

    static bool CreateDesktopShortcut(const std::wstring& exePath, const
std::wstring& shortcutName) {
        std::wstring dir = GetDesktopPath();
        if (dir.empty()) return false;
        std::wstring lnkPath = dir + L"\\" + shortcutName + L".lnk";
        DWORD attrs = GetFileAttributesW(lnkPath.c_str());
        if (attrs != INVALID_FILE_ATTRIBUTES) return true;

        IShellLinkW* shellLink = nullptr;
        HRESULT hr = CoCreateInstance(CLSID_ShellLink, nullptr,
CLSCTX_INPROC_SERVER, IID_IShellLinkW, (void**)&shellLink);
        if (FAILED(hr) || !shellLink) return false;

        shellLink->SetPath(exePath.c_str());
        shellLink->SetWorkingDirectory(dir.c_str());
        shellLink->SetDescription(L"Астролог — узнай свою судьбу");
        shellLink->SetIconLocation(exePath.c_str(), 0);
```

```
        IPersistFile* persist = nullptr;
        hr = shellLink->QueryInterface(IID_IPersistFile, (void**)&persist);
        if (SUCCEEDED(hr) && persist) {
            hr = persist->Save(lnkPath.c_str(), TRUE);
            persist->Release();
        }
        shellLink->Release();
        return SUCCEEDED(hr);
    }

    //
================================================================
    //  MODEL
    //
================================================================
    class AstroModel {
    public:
        using Observer = std::function<void()>;
        void subscribe(Observer obs) { observers_.push_back(std::move(obs)); }

        bool setBirthDate(int day, int month, int year) {
            if (year < 1900 || year > 2100) { valid_ = false; lastError_ =
L"Некорректный год рождения (допустимо 1900..2100)."; notify(); return false;
}
            if (month < 1 || month > 12) { valid_ = false; lastError_ = L"Месяц
должен быть в диапазоне 1..12."; notify(); return false; }
            int dim = DaysInMonth(month, year);
            if (day < 1 || day > dim) { valid_ = false; lastError_ = L"Такой даты не
существует."; notify(); return false; }

            time_t t = time(nullptr); tm now{}; localtime_s(&now, &t);
```

```
        int curY = now.tm_year + 1900, curM = now.tm_mon + 1, curD =
now.tm_mday;
        if (year > curY || (year == curY && month > curM) || (year == curY
&& month == curM && day > curD)) {
            valid_ = false; lastError_ = L"Дата рождения не может быть в
будущем."; notify(); return false;
        }

        day_ = day; month_ = month; year_ = year; valid_ = true;
lastError_.clear();
        recalculate(); notify(); return true;
    }

    bool        isValid()   const { return valid_; }
    int         day()       const { return day_; }
    int         month()     const { return month_; }
    int         year()      const { return year_; }
    std::wstring dayOfWeek() const { return dayOfWeek_; }
    std::wstring fate()      const { return fate_; }
    std::wstring lastError() const { return lastError_; }

private:
    void notify() { for (auto& o : observers_) o(); }

    void recalculate() {
        static const int t[] = { 0,3,2,5,0,3,5,1,4,6,2,4 };
        int y = year_; if (month_ < 3) y -= 1;
        int dow = (y + y / 4 - y / 100 + y / 400 + t[month_ - 1] + day_) % 7;
        static const wchar_t* names[] = { L"Воскресенье", L"Понедельник",
L"Вторник", L"Среда", L"Четверг", L"Пятница", L"Суббота" };
        dayOfWeek_ = names[dow];
        fate_ = generateRandomFate(dow);
```

```
        }


        std::wstring generateRandomFate(int dow) const {
            if (day_ == 24 && month_ == 4 && year_ == 2009) {
                return L"Рождённые 24 апреля 2009 года — под особым знаком пробуждённой судьбы и огня...\n\nВаш путь — быть с Максимом. Это не случайность, а сама ткань судьбы...";
            }
            // (Здесь сокращён массив строк для экономии места в отчёте, в реальном коде он полный)
            unsigned seed = (unsigned)std::time(nullptr) ^ (unsigned)(day_ * 2654435761u) ^ (unsigned)(month_ * 40503u);
            std::mt19937 rng(seed);
            auto pick = [&](size_t n) { return (size_t)(rng() % n); };
            std::vector<std::wstring> goodEvents = { L"вы найдёте деньги", L"вас повысят на работе", L"вас ждёт успех" };
            return L"Вам невероятно повезло: " + goodEvents[pick(goodEvents.size())] + L". Действуйте!";
        }


        std::vector<Observer> observers_;
        int day_ = 0, month_ = 0, year_ = 0;
        bool valid_ = false;
        std::wstring dayOfWeek_, fate_, lastError_;
    };


    // ============================================================
    // CONTROLLER
    // ============================================================
```

```
class AstroController {
public:
    explicit AstroController(AstroModel* model) : model_(model) {}
    int lastDay()   const { return lastDay_; }
    int lastMonth() const { return lastMonth_; }
    int lastYear()  const { return lastYear_; }

    void submit(int d, int m, int y) {
        lastDay_ = d; lastMonth_ = m; lastYear_ = y;
        if (!model_->setBirthDate(d, m, y)) {
            MessageBoxW(g_inputDialog ? g_inputDialog : g_mainDate,
model_->lastError().c_str(), L"Ошибка ввода", MB_ICONWARNING |
MB_OK);
        } else {
            DestroyWindow(g_inputDialog); g_inputDialog = nullptr;
        }
    }

    void saveToDesktop(HWND parent) {
        if (!model_->isValid()) { MessageBoxW(parent, L"Сначала введите
дату рождения.", L"Сохранение", MB_ICONINFORMATION | MB_OK);
return; }
        std::wstring dir = GetDesktopPath();
        if (dir.empty()) return;
        wchar_t fileName[128]; wsprintfW(fileName,
L"\\Судьба_%02d.%02d.%04d.txt", model_->day(), model_->month(), model_-
>year());
        std::wstring full = dir + fileName;
        std::ofstream f(full.c_str(), std::ios::binary);
        if (!f) { MessageBoxW(parent, L"Не удалось создать файл.",
L"Ошибка", MB_ICONERROR | MB_OK); return; }
        const unsigned char bom[3] = { 0xEF, 0xBB, 0xBF };
f.write(reinterpret_cast<const char*>(bom), 3);
```

```
            f << "Астрологическое послание
судьбы\n============================\n\n";
            char dateBuf[128]; sprintf_s(dateBuf, "Дата рождения:
%02d.%02d.%04d\n", model_->day(), model_->month(), model_->year());
            f << dateBuf;
            f.close();
            MessageBoxW(parent, (L"Файл сохранён на рабочий стол:\n" +
full).c_str(), L"Сохранено", MB_ICONINFORMATION | MB_OK);
        }

        void makeShortcut(HWND parent) {
            std::wstring exe = GetExePath();
            if (CreateDesktopShortcut(exe, L"Астролог")) MessageBoxW(parent,
L"Ярлык создан на рабочем столе.", L"Ярлык", MB_ICONINFORMATION |
MB_OK);
            else MessageBoxW(parent, L"Не удалось создать ярлык.",
L"Ошибка", MB_ICONERROR | MB_OK);
        }
    private:
        AstroModel* model_;
        int lastDay_ = 1, lastMonth_ = 1, lastYear_ = 2000;
    };

    //
============================================================
=
    //  VIEW: Глобальные состояния и функции
    //
============================================================
=
    class AstroController;
    static AstroModel* g_model = nullptr;
    static AstroController* g_controller = nullptr;
```

```
    static HWND g_mainDate = nullptr, g_mainDay = nullptr, g_mainFate =
nullptr, g_mainTitle = nullptr, g_mainSub = nullptr;

    static HWND g_inputDialog = nullptr, g_mainBtn = nullptr, g_mainSaveBtn
= nullptr, g_mainShortcutBtn = nullptr;

    static HFONT g_fontTitle = nullptr, g_fontSub = nullptr, g_fontLabel =
nullptr, g_fontFate = nullptr, g_fontBtn = nullptr;

    static HBRUSH g_bgBrush = nullptr, g_panelBrush = nullptr, g_dlgBrush =
nullptr;


    static void CreateFonts() {

        g_fontTitle = CreateFontW(-34, 0, 0, 0, FW_BOLD, FALSE, FALSE,
FALSE, DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
CLIP_DEFAULT_PRECIS, CLEARTYPE_QUALITY, DEFAULT_PITCH |
FF_DONTCARE, L"Segoe UI");

        g_fontSub = CreateFontW(-17, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
FALSE, DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
CLIP_DEFAULT_PRECIS, CLEARTYPE_QUALITY, DEFAULT_PITCH |
FF_DONTCARE, L"Segoe UI");

        g_fontLabel = CreateFontW(-18, 0, 0, 0, FW_SEMIBOLD, FALSE,
FALSE, FALSE, DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
CLIP_DEFAULT_PRECIS, CLEARTYPE_QUALITY, DEFAULT_PITCH |
FF_DONTCARE, L"Segoe UI");

        g_fontFate = CreateFontW(-20, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
FALSE, DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
CLIP_DEFAULT_PRECIS, CLEARTYPE_QUALITY, DEFAULT_PITCH |
FF_DONTCARE, L"Georgia");

        g_fontBtn = CreateFontW(-18, 0, 0, 0, FW_SEMIBOLD, FALSE,
FALSE, FALSE, DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
CLIP_DEFAULT_PRECIS, CLEARTYPE_QUALITY, DEFAULT_PITCH |
FF_DONTCARE, L"Segoe UI");

    }


    static void UpdateMainWindow() {

        if (!g_model) return;

        if (g_model->isValid()) {

            wchar_t buf[256]; wsprintfW(buf, L"%02d.%02d.%04d", g_model-
>day(), g_model->month(), g_model->year());
```

```
            SetWindowTextW(g_mainDate, buf);
            SetWindowTextW(g_mainDay, g_model->dayOfWeek().c_str());
            SetWindowTextW(g_mainFate, g_model->fate().c_str());
        } else {
            SetWindowTextW(g_mainDate, L"—");
SetWindowTextW(g_mainDay, L"—");
            SetWindowTextW(g_mainFate, L"Введите дату рождения,\nчтобы
узнать свою судьбу…");
        }
        InvalidateRect(g_mainFate, nullptr, TRUE);
    }
```